Algorithmique & Programmation

Corwin FEVRE corwin.fevre@eigsi.fr

FISA 3 — EIGSI



Inspiré du cours de M. Thomas RAIMBAULT

Partie V

Structures avancées I

1. Préambule
2. Les structures avancées

1 Préambule
- Bilan de ce que l'on a vu

2 Les structures avancées
- Les piles
- Les files
- Les arbres binaires
- Les graphes

Problème

Selon le problème à traiter, le choix d'une structure de données adaptée est primordial en termes de performance (algorithmique).

- Besoin de traiter des taches dans un ordre précis
- Besoin de parcourir un espace de recherche (une carte...)
- Besoin d'étudier un flux
- ...

1 Préambule
- Bilan de ce que l'on a vu

2 Les structures avancées
- Les piles
- Les files
- Les arbres binaires
- Les graphes

Nous avons vu qu'il existe plusieurs structures de données en Python :

- **Les listes :** travailler sur des données de tout type dynamiquement
- **Les ensembles :** travailler sur des données uniques et de même type
- **Les tuples :** verrouiller et transmettre des résultats entre les fonctions, permuter des valeurs
- **Les dictionnaires :** associer des valeurs à des clés pour permettre un indiçage plus intelligent et intuitif

Nous avons vu qu'il existe plusieurs structures de données en Python :

- **Les listes :** travailler sur des données de tout type dynamiquement
- **Les ensembles :** travailler sur des données uniques et de même type
- **Les tuples :** verrouiller et transmettre des résultats entre les fonctions, permuter des valeurs
- **Les dictionnaires :** associer des valeurs à des clés pour permettre un indiçage plus intelligent et intuitif

Bien que ces structures aient été jusque là suffisantes, il est nécessaire d'utiliser d'autres structures plus complexes pour résoudre d'autres types de problèmes.

✎ Ces structures avancées sont construites à partir des structures primitives de Python, en association avec des fonctions qui leurs sont propres !

Les structures avancées

Dans ce chapitre nous allons étudier :

- Les piles
- Les files
- Les arbres
- Les graphes

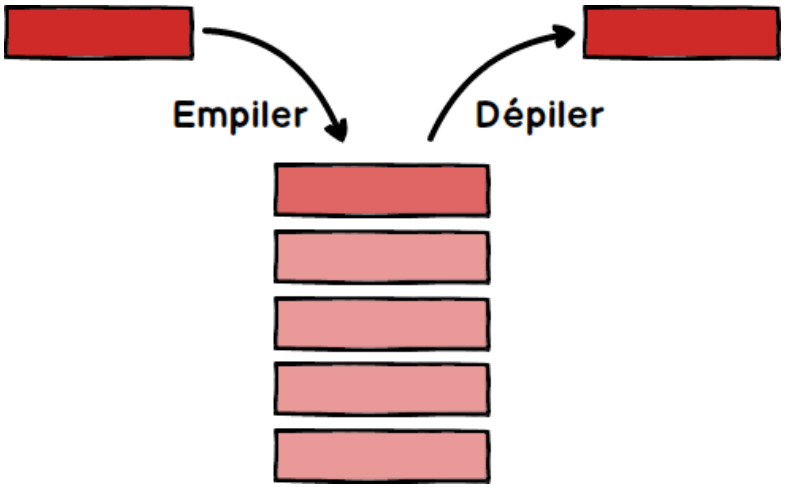
1 Préambule
- Bilan de ce que l'on a vu

2 Les structures avancées
- Les piles
- Les files
- Les arbres binaires
- Les graphes

1 Préambule
- Bilan de ce que l'on a vu

2 Les structures avancées
- Les piles
- Les files
- Les arbres binaires
- Les graphes

Les piles





Pile → Liste LIFO : Last-in, First-out (Dernier entré, premier sorti)

Les piles - Les fonctions

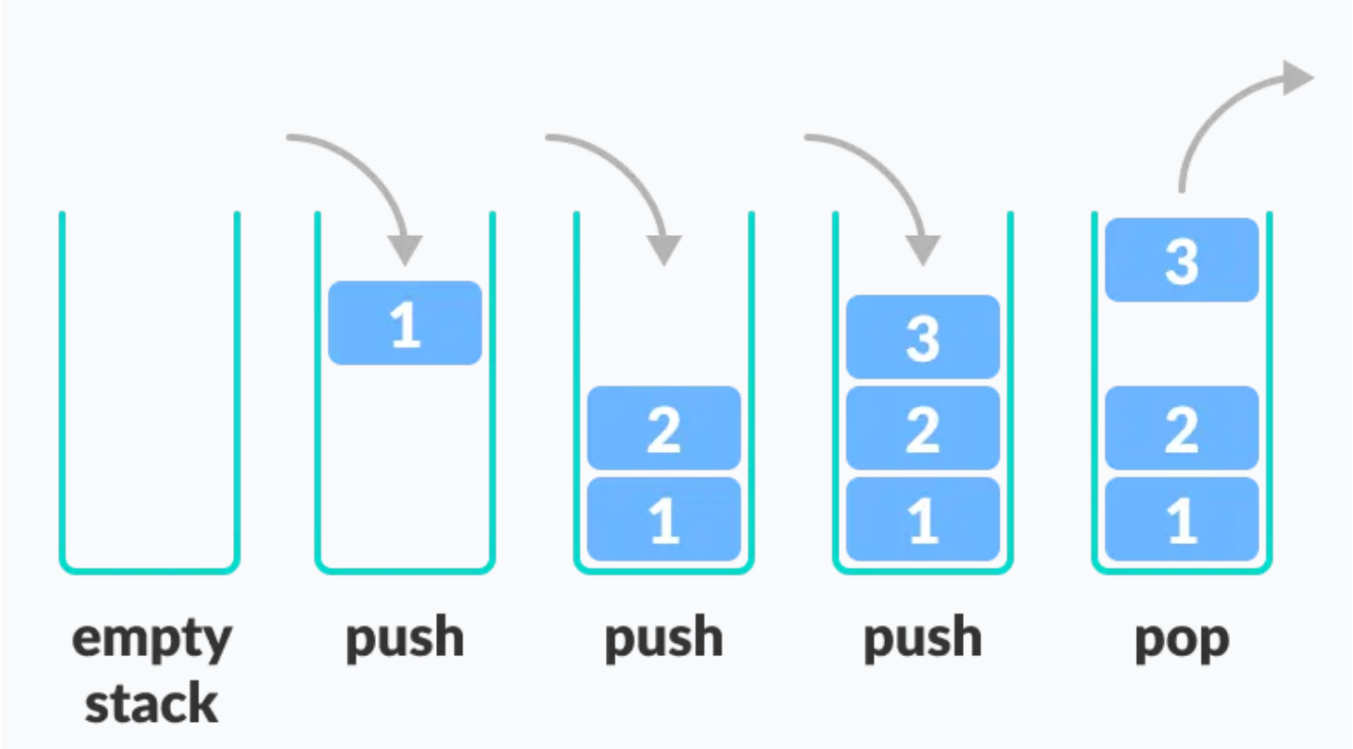
Les principales fonctions d'une pile (stack) :

- **Push :** ajouter un élément en haut de la pile
- **Pop :** retirer un élément du haut de la pile
- **IsEmpty :** vérifier si la pile est vide
- **Peek :** récupérer la valeur en haut de la pile sans la retirer

Quelques applications :

- Inverser un mot : mettre toutes les lettres d'un mot dans une file, puis les retirer une à une
- Les compilateurs : pour le calcul des expressions, pour les appels de fonction
- Les navigateurs web : sauvegarde des pages visitées, bouton retour...
- Comptabilité : évaluer le coût des stocks vendus et des stocks restants

Les piles - Exemple

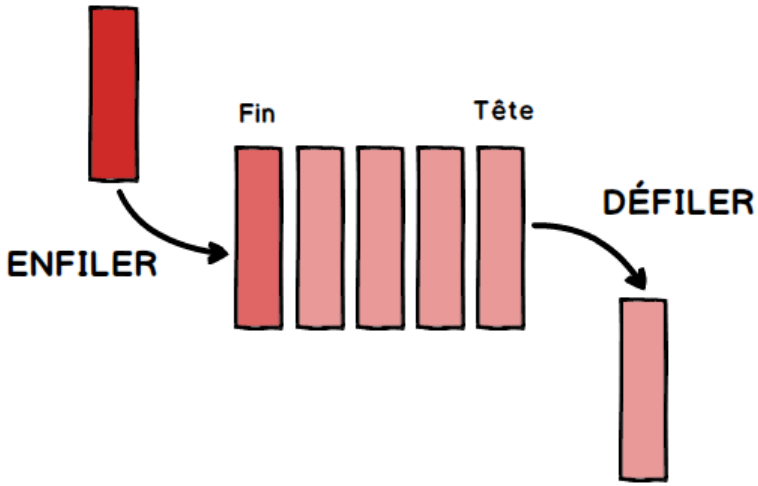


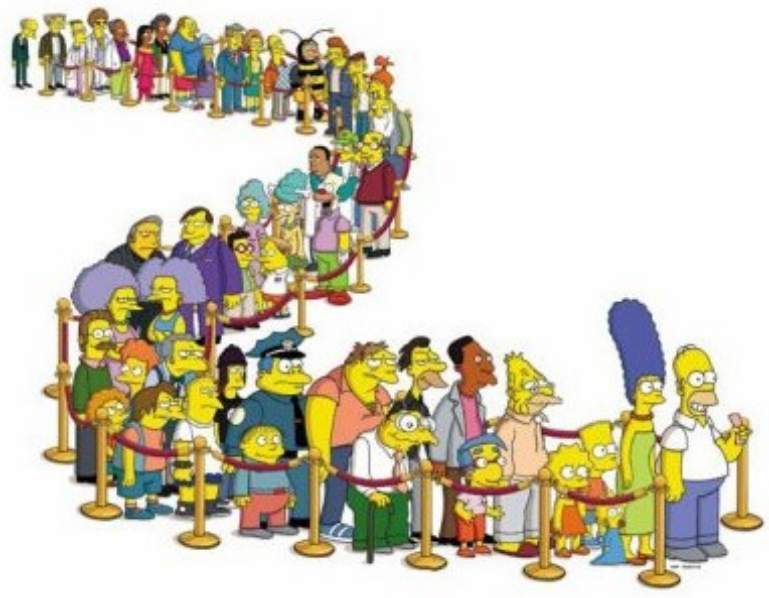
✎ Approche verticale !

1 Préambule
- Bilan de ce que l'on a vu

2 Les structures avancées
- Les piles
- Les files
- Les arbres binaires
- Les graphes

Les files





File → Liste FIFO : First-in, First-out (Premier entré, premier sorti)

Les files - Les fonctions

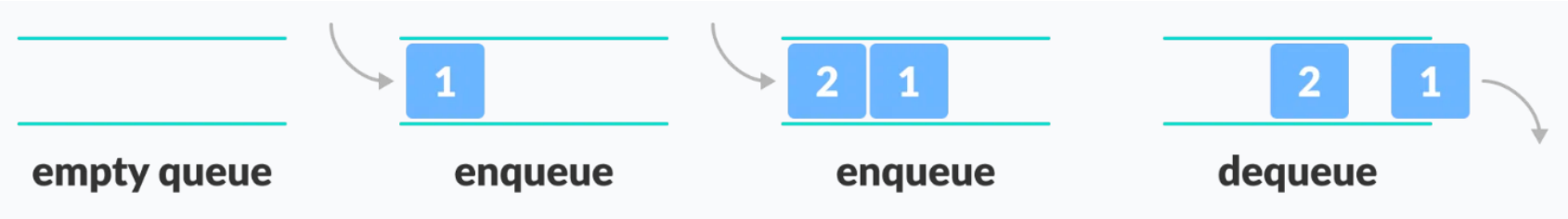
Les principales fonctions d'une file (queue) :

- **Enqueue :** ajouter un élément a la fin de la file
- **Dequeue :** retirer un élément à la tête de la file
- **IsEmpty :** vérifier si la file est vide
- **Peek :** récupérer la valeur de la tête de la file sans la retirer

Quelques applications :

- En général : Problématiques premier arrivé, premier servi
- Ordonnancement des tâches d'un processeur, des disques
- Théorie des files d'attente

Les files - Exemple



✎ Approche horizontale !

1 Préambule
- Bilan de ce que l'on a vu

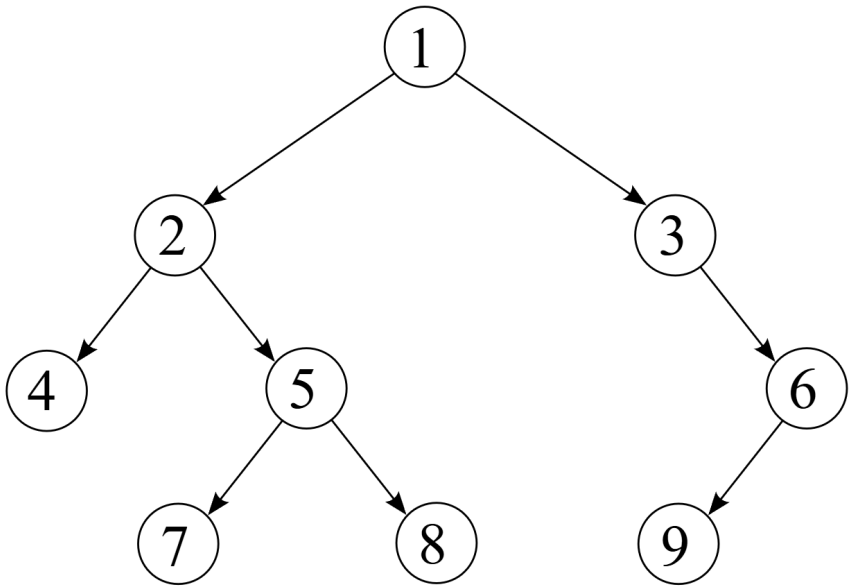
2 Les structures avancées
- Les piles
- Les files
- Les arbres binaires
- Les graphes

Les arbres binaires

Ensemble hiérarchisé de nœuds

Chaque nœud possède :

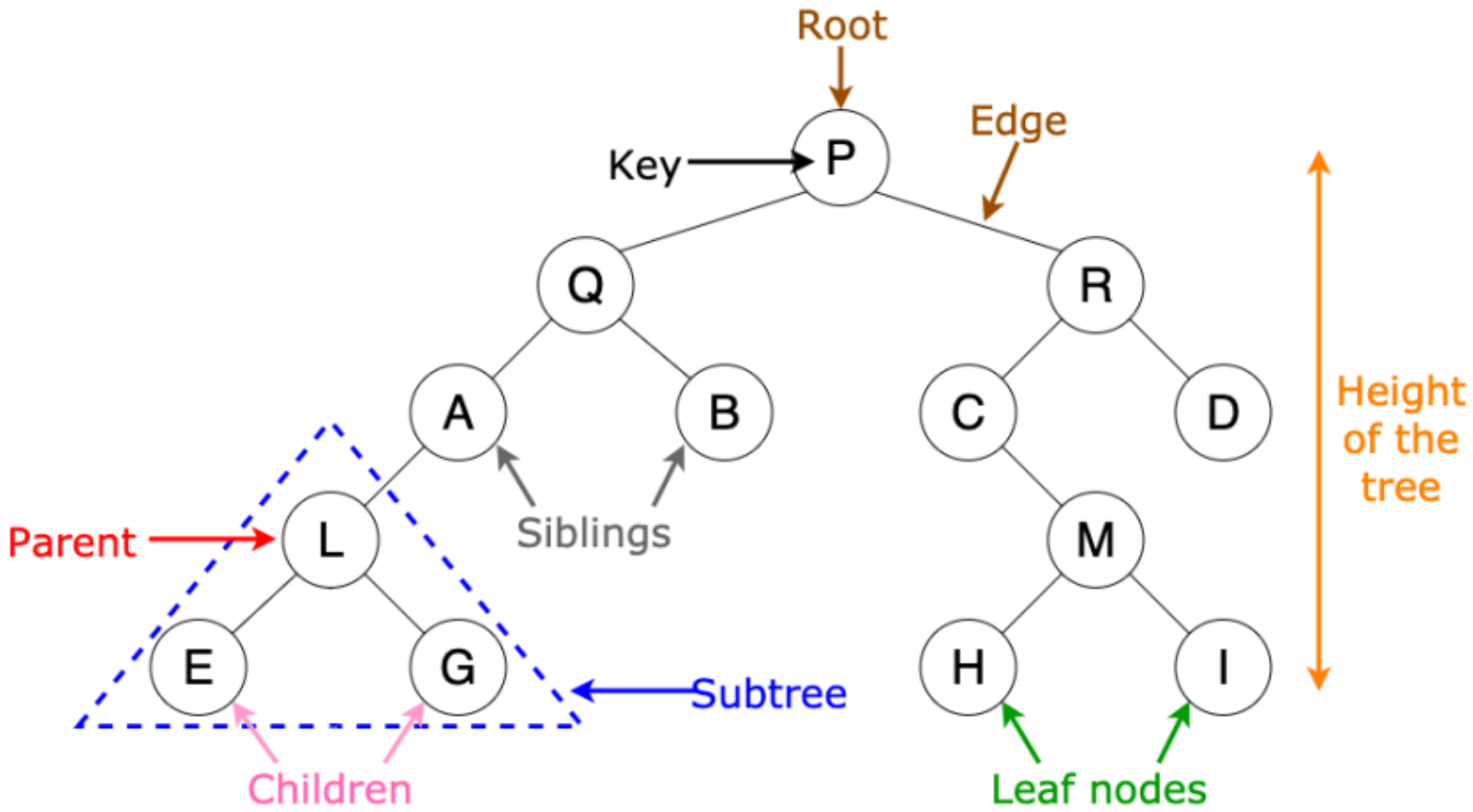
- un contenu
- un unique père (sauf racine)
- au plus un fils gauche
- au plus un fils droit



Caractéristiques :

- hauteur, ici 4 (h = 1 si un seul nœud)
- nb max de nœuds = $2^h - 1 \Leftrightarrow$ arbre complet
- nb max de feuilles = 2^{h-1}

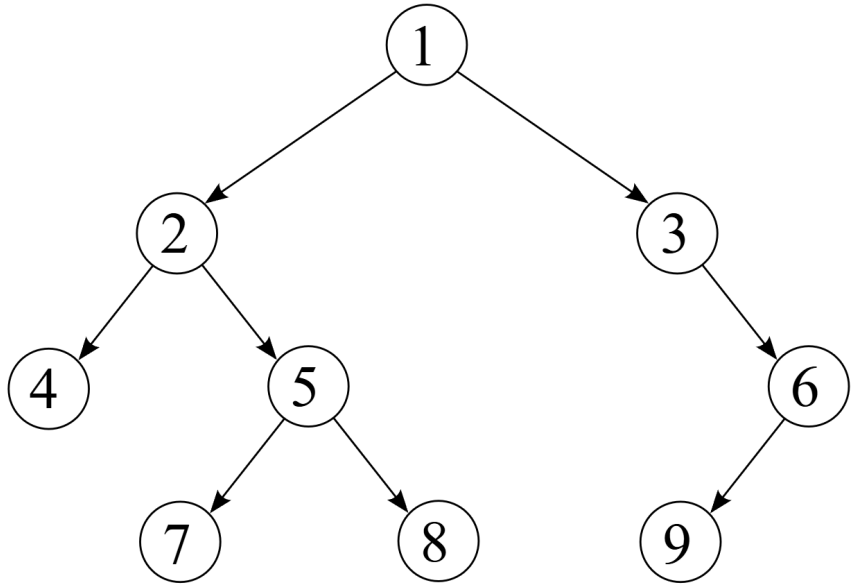
Les arbres binaires - Vocabulaire



Les arbres binaires

Opérations classiques :

- ajout / suppression d'un nœud
- parcours / recherche (en profondeur, en largeur)
- ...



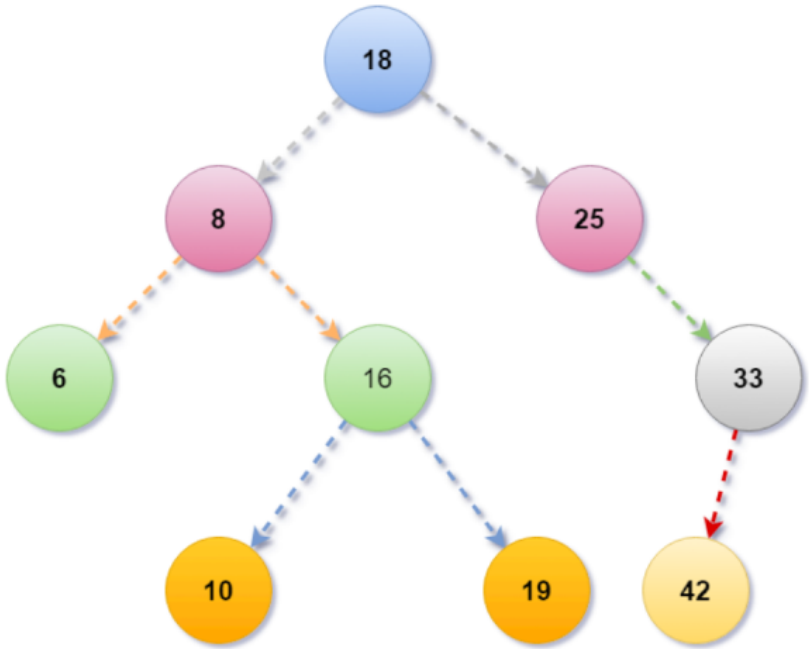
Arbre binaire de recherche

Arbre binaire de recherche (ABR) :

- arbre binaire où toutes les valeurs sont ordonnées

→ pour chaque valeur :

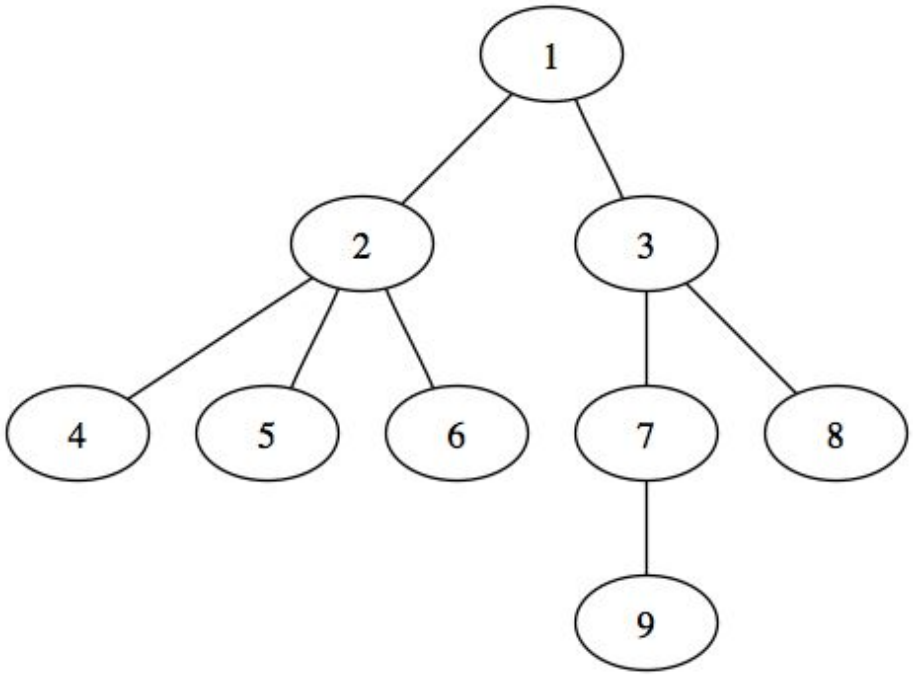
- toutes les valeurs du sous-arbre de gauche lui sont inférieures
- toutes les valeurs du sous-arbre de droite lui sont supérieures



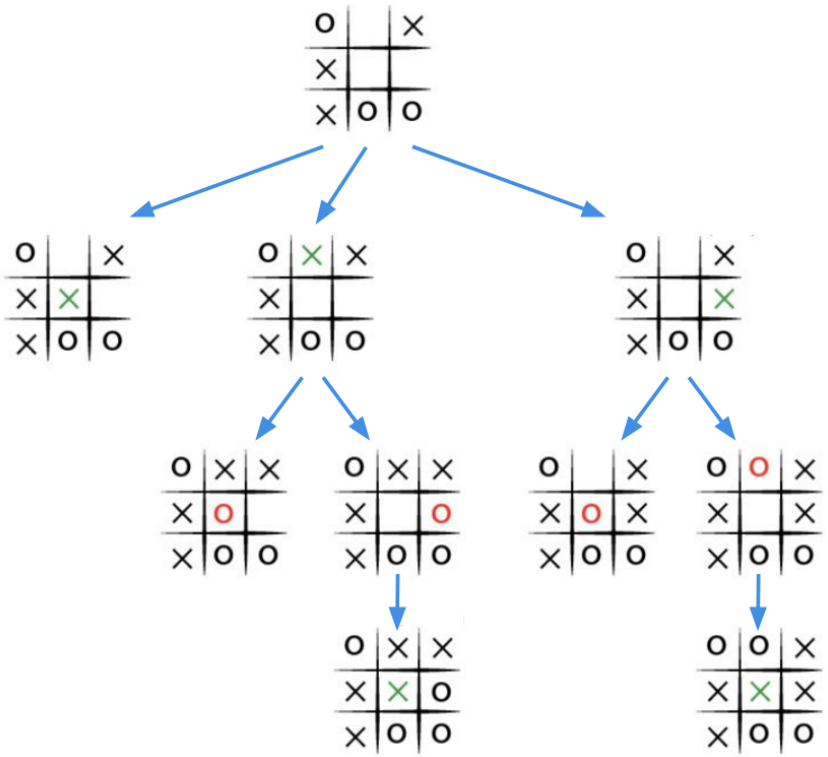
Arbre n-aire

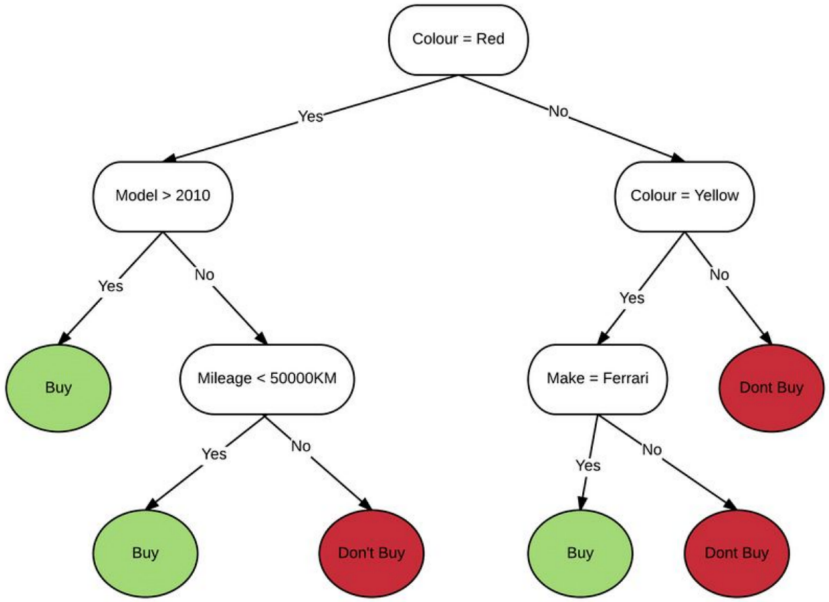
Arbre n-aire :

- Chaque nœud possède :
  - un contenu
  - un unique père (sauf racine)
  - un ensemble de fils (n au plus)



Exemple utilisation d'arbres





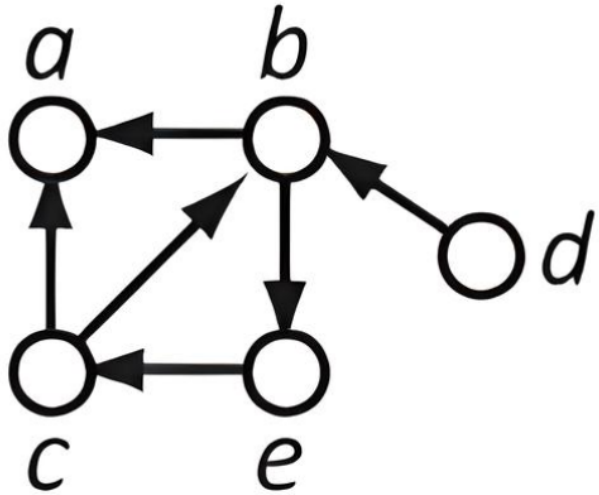
1 Préambule
- Bilan de ce que l'on a vu

2 Les structures avancées
- Les piles
- Les files
- Les arbres binaires
- Les graphes

Les graphes - Structures relationnelles dynamiques

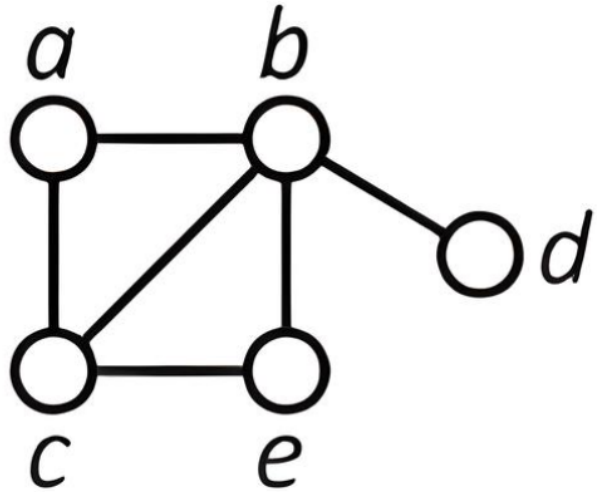
Graphes orientés :

- un nœud possède :
  - un contenu
  - un ensemble d'arcs (entrants et/ou sortants)
- un arc relie un nœud source à un nœud de destination

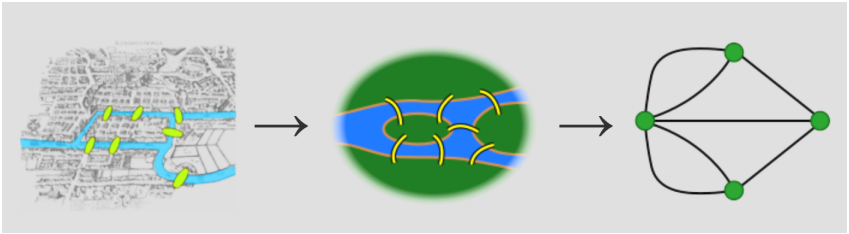


Graphes non orientés :

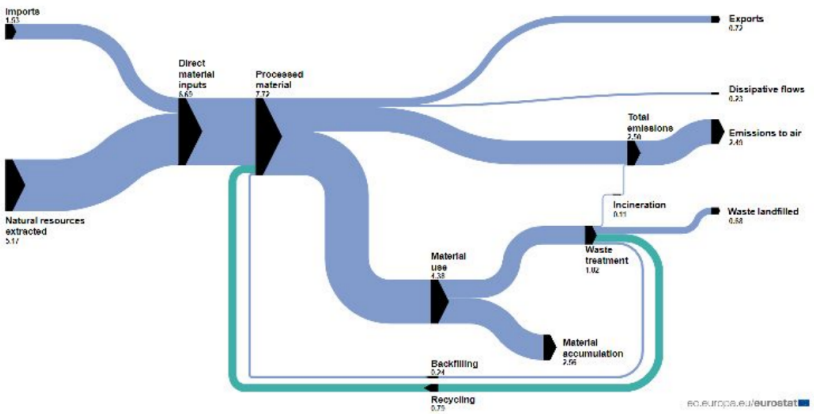
- un nœud possède :
  - un contenu
  - un ensemble d'arêtes
- une arête relie deux nœuds



Exemple d'utilisation des graphes









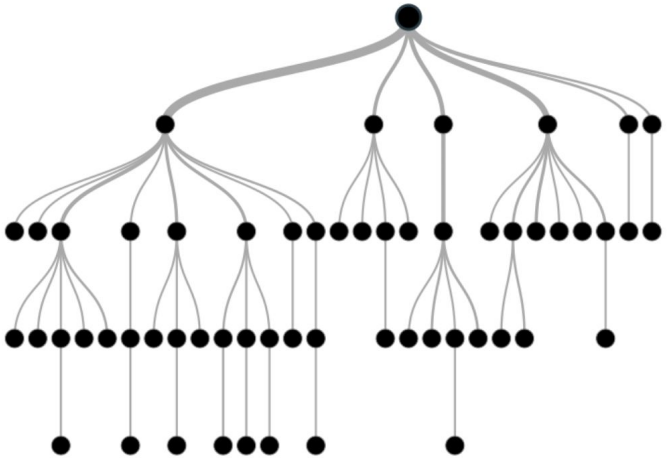
Structures de données

➜ algorithmes

EIGSI
ÉCOLE D'INGÉNIEURS
LA ROCHELLE • CASABLANCA

Thomas Raimbault

Algorithmes pour arbres



Parcours d’un arbre

Deux opérations de bases :

- parcours en largeur

versus

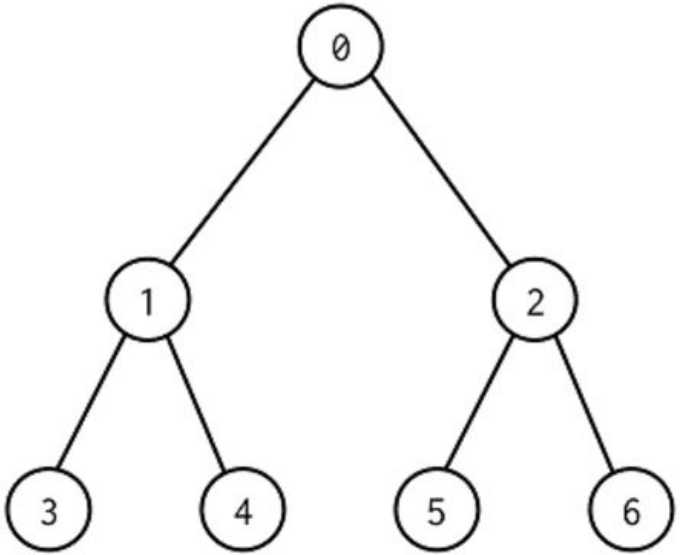
- parcours en profondeur (pré-fixé, infixé, post-fixé)

Usages :

- accès aux nœuds d’un arbre
- recherche d’un élément
- recherche d’un chemin entre deux éléments
- …

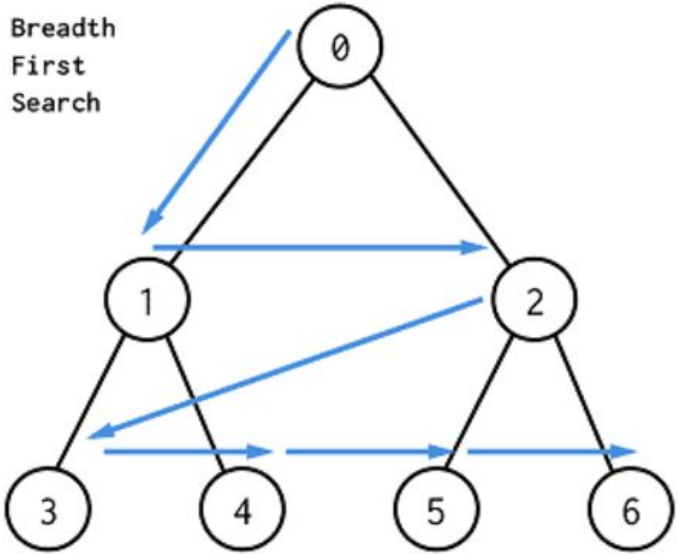
Parcours en largeur

- Visiter tous les fils, puis pour chaque fils faire de même, *etc*.



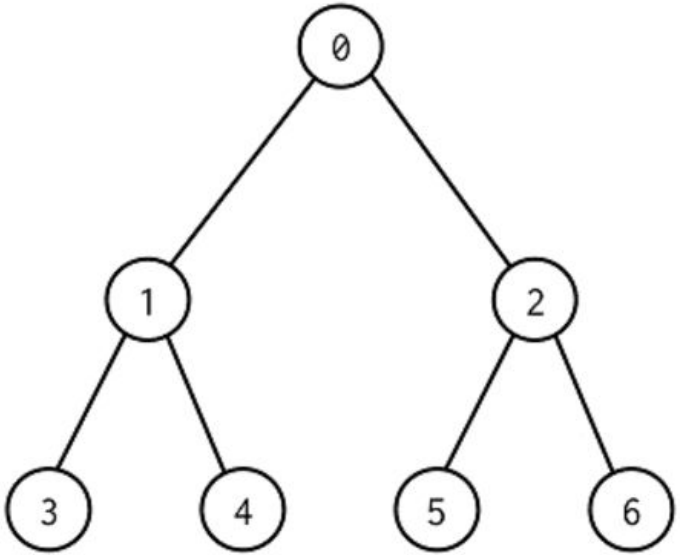
Parcours en largeur

- Visiter tous les fils, puis *idem* pour chaque fils, etc.
- Ici, parcours effectué depuis la racine : 0, 1, 2, 3, 4, 5, 6



Parcours en profondeur

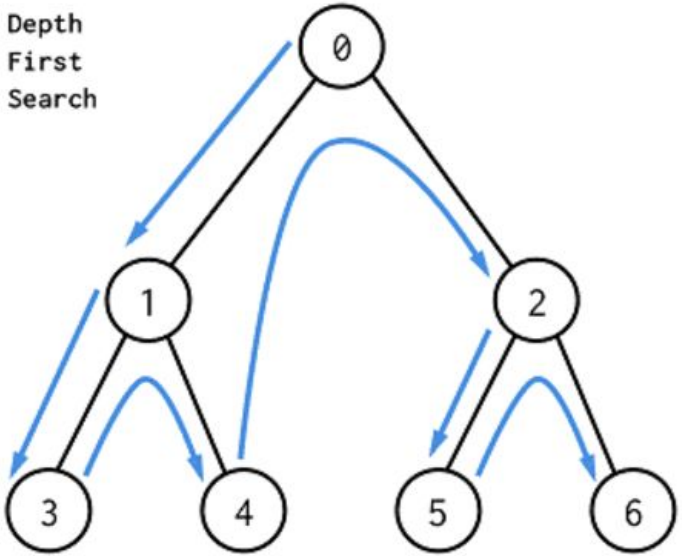
- Visiter en poursuivant un chemin vers les fils tant que c’est possible (jusqu'à un cul-de-sac), puis revenir en arrière, et poursuivre, …



Parcours en profondeur

- Visiter en poursuivant un chemin vers les fils tant que c'est possible (jusqu'à un cul-de-sac), puis revenir en arrière, et poursuivre, …

- Ici, parcours effectué depuis la racine : 0, 1, 3, 4, 2, 5, 6

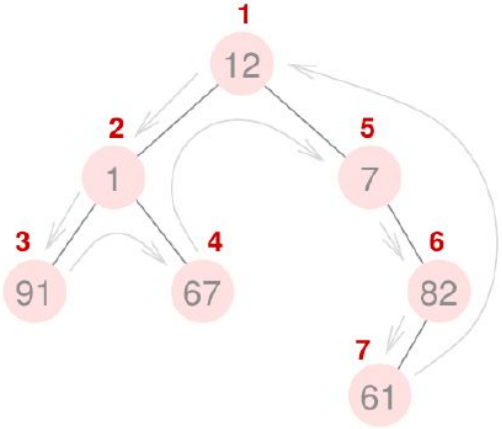


Parcours en profondeur

- 3 parcours « en profondeur » :
  - préfixé (celui vu précédemment)
  - infixé
  - post-fixé

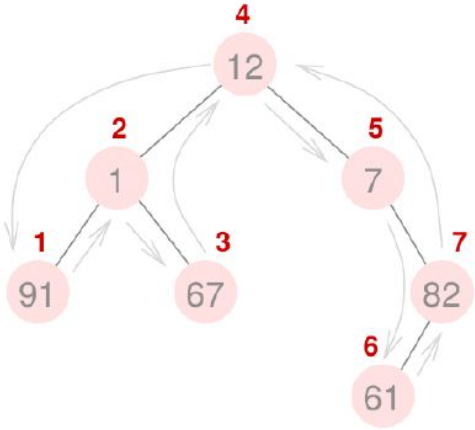


parcours pré-fixé



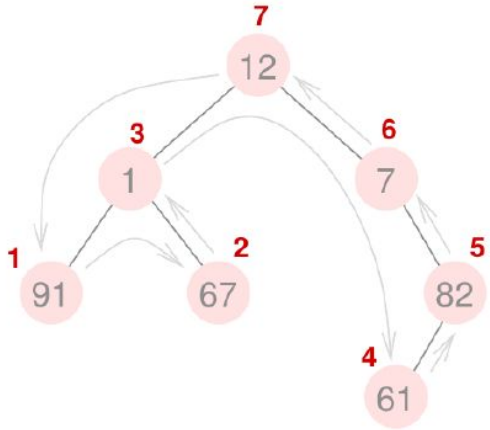
12 1 91 67 7 82 61

parcours infixé



91 1 67 12 7 61 82

parcours post-fixé



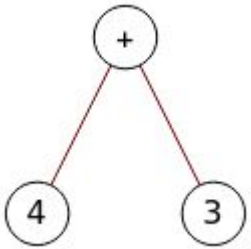
91 67 1 61 82 7 12

Parcours en profondeur

- 3 parcours « en profondeur » :
  - préfixé (celui vu précédemment)
  - infixé
  - post-fixé



- Écritures d'une expression algébrique :
  - écriture préfixé : + 4 3
  - écriture infixé : 4 + 3 (mais valable que pour opérateurs à 2 opérandes)
  - écriture post-fixé : 4 3 + (= « notation polonaise inverse »)



Algorithme de parcours en largeur dans le détail

Pseudo-code itératif (cf. wikipedia) :

- nécessite une **File** (*Queue*)

Input: A graph G and a *starting vertex root* of G
Output: Goal state. The *parent* links trace the shortest path back to *root*

```
 1  procedure BFS(G, root) is
 2      let Q be a queue
 3      label root as explored
 4      Q.enqueue(root)
 5      while Q is not empty do
 6          v := Q.dequeue()
 7          if v is the goal then
 8              return v
 9          for all edges from v to w in G.adjacentEdges(v) do
10              if w is not labeled as explored then
11                  label w as explored
12                  w.parent := v
13                  Q.enqueue(w)
```

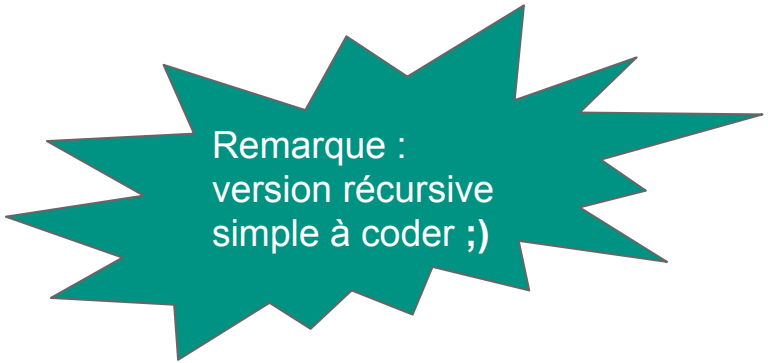
Illustration : https://www.youtube.com/watch?v=4-TE4Avkmec&t=39s

Algorithme de parcours en profondeur dans le détail

Pseudo-code itératif (cf. wikipedia) :

- nécessite une **Pile** (*Stack*)

```
procedure DFS_iterative(G, v) is
    let S be a stack
    S.push(iterator of G.adjacentEdges(v))
    while S is not empty do
        if S.peek().hasNext() then
            w = S.peek().next()
            if w is not labeled as discovered then
                label w as discovered
                S.push(iterator of G.adjacentEdges(w))
        else
            S.pop()
```



Illustrations itératives :

- préfixé : https://www.youtube.com/watch?v=zwTQ9oKZ8lE&t=89s
- infixé : https://www.youtube.com/watch?v=WI1mbEmz0oI&t=102s
- post-fixé : https://www.youtube.com/watch?v=LUwHA0kNxDQ&t=136s